Лабораторная работа № 6

Разработка REST API с использованием ASP.NET Core Minimal API и публикация на Render

Дисциплина: «Web-технологии»
Специальность: «Управление информационными ресурсами»
Продолжительность: 4 академических часа
Максимальная оценка: 21 балл

---

1. Цель работы

Освоить разработку REST API с использованием **ASP.NET Core Minimal API**, научиться создавать HTTP-эндпоинты, передавать и получать данные в формате JSON, использовать параметры маршрута и тела HTTP-запроса, возвращать корректные HTTP-статусы и публиковать приложение с использованием **Docker и Render**.

В ходе выполнения лабораторной работы:

- создать проект ASP.NET Core Minimal API;
- разработать модель предметной области;
- реализовать хранение данных в памяти;
- реализовать REST API;
- реализовать операции `GET`, `POST`, `PUT`, `DELETE`;
- использовать JSON;
- использовать параметры маршрута и query-параметры;
- использовать HTTP-статусы;
- добавить описание API с использованием OpenAPI;
- создать Dockerfile;
- разместить проект в Git-репозитории;
- опубликовать приложение на Render;
- проверить работу API по публичному HTTPS-адресу.

---

2. Исходные условия

Использовать ту же предметную область и модель данных, которые были разработаны в предыдущих лабораторных работах.

Например:

Предметная область	Основные сущности
Интернет-магазин	Product, Category, Order, Customer
Библиотека	Book, Author, Reader, LoanRecord

Предметная область	Основные сущности
Автопрокат	Car, Client, RentalContract, Manager
Банк	Account, Transaction, BankClient, Loan
Медицинский центр	Doctor, Patient, Appointment, MedicalRecord
Университет	Student, Course, Teacher, Grade
Гостиница	Room, Guest, Reservation, Hotel
Авиакомпания	Flight, Passenger, Ticket, Airport
Доставка еды	Dish, Order, Courier, Restaurant
Спортивный клуб	Trainer, Member, Subscription, Workout
Управление IT-проектами	Task, Project, Developer, Sprint
Склад и логистика	Cargo, Warehouse, Driver, Shipment
Музыкальный сервис	Track, Album, Artist, Playlist
Недвижимость	Property, Agent, Deal, Client
Игровые турниры	Player, Team, Tournament, Match

Использовать ту же основную сущность, которая использовалась в WT-4 и WT-5.

---

3. Ограничения

В лабораторной работе:

- не использовать PostgreSQL;
- не использовать другие СУБД;
- не использовать Entity Framework Core;
- не использовать ORM;
- хранить данные в памяти приложения;
- использовать коллекцию C# в качестве временного хранилища;
- использовать JSON для передачи данных;
- использовать Docker для подготовки приложения к публикации;
- опубликовать приложение на Render.

---

4. Выполнение работы

Блок 1. Создать проект ASP.NET Core Minimal API

Уровень 1 — 1 балл

Создать проект:

```
dotnet new web -n wt-lab6-6minimalapi-ivanov
```

Заменить `ivanov` на свою фамилию.

Перейти в каталог проекта:

```
cd wt-lab6-6minimalapi-ivanov
```

Запустить приложение:

```
dotnet run
```

Проверить успешность запуска приложения.

Уровень 2 — 2 балла

Создать endpoint:

```
GET /
```

Использовать следующий код:

```
app.MapGet("/", () => "Minimal API работает!");
```

Запустить приложение.

Открыть endpoint в браузере.

Проверить получение текста:

```
Minimal API работает!
```

Уровень 3 — 3 балла

Создать endpoint:

```
GET /api/info
```

Реализовать возврат информации о приложении:

```
app.MapGet("/api/info", () =>
{
    return new
    {
        Application = "Web Technologies",
        Technology = "ASP.NET Core Minimal API",
        Version = "1.0"
    };
});
```

Запустить приложение.

Открыть endpoint в браузере.

Проверить получение данных в формате JSON.

Объяснить преобразование объекта C# в JSON.

---

5. Блок 2. Создать модель и хранилище данных

5.1. Создать модель

Создать каталог:

```
Models
```

Создать файл:

```
Models/Product.cs
```

Создать класс предметной области.

Пример:

```
namespace wt_lab6_6minimalapi_ivanov.Models;

public class Product
{
    public int Id { get; set; }

    public string Name { get; set; } = "";
```

```
    public string Category { get; set; } = "";

    public decimal Price { get; set; }

    public bool IsAvailable { get; set; }
}
```

Адаптировать класс к выбранной предметной области.

5.2. Создать хранилище

Создать каталог:

```
Data
```

Создать файл:

```
Data/ProductStore.cs
```

Создать статический класс с коллекцией объектов.

Пример:

```
using wt_lab6_6minimalapi_ivanov.Models;

namespace wt_lab6_6minimalapi_ivanov.Data;

public static class ProductStore
{
    public static List<Product> Products { get; } =
        new()
        {
            new Product
            {
                Id = 1,
                Name = "Ноутбук",
                Category = "Компьютеры",
                Price = 2500,
                IsAvailable = true
            },

            new Product
            {
                Id = 2,
                Name = "Мышь",
                Category = "Аксессуары",
```

```
                Price = 80,
                IsAvailable = true
            }
        };
}
```

Добавить не менее пяти объектов.

Использовать одну коллекцию для всех API-операций.

Уровень 1 — 1 балл

Создать класс основной сущности.

Добавить идентификатор и не менее двух дополнительных свойств.

Уровень 2 — 2 балла

Добавить не менее четырех свойств.

Создать не менее пяти объектов.

Уровень 3 — 3 балла

Создать отдельный класс `ProductStore`.

Создать общую коллекцию объектов.

Использовать одну коллекцию для всех API-операций.

---

6. Блок 3. Реализовать GET-запросы и маршрутизацию

6.1. Реализовать получение списка

Создать endpoint:

```
GET /api/products
```

Реализовать получение всех объектов:

```
app.MapGet("/api/products", () =>
{
    return Results.Ok(ProductStore.Products);
});
```

6.2. Реализовать получение объекта по идентификатору

Создать endpoint:

```
GET /api/products/{id}
```

Реализовать поиск объекта:

```
app.MapGet("/api/products/{id}", (int id) =>
{
    var product = ProductStore.Products
        .FirstOrDefault(x => x.Id == id);

    if (product == null)
        return Results.NotFound();

    return Results.Ok(product);
});
```

Проверить существующий идентификатор.

Проверить несуществующий идентификатор.

Получить статус:

```
404 Not Found
```

при отсутствии объекта.

6.3. Реализовать фильтрацию

Добавить query-параметр:

```
GET /api/products?search=ноут
```

Реализовать фильтрацию:

```
app.MapGet("/api/products", (string? search) =>
{
    var products = ProductStore.Products.AsEnumerable();

    if (!string.IsNullOrWhiteSpace(search))
    {
        products = products.Where(x =>
            x.Name.Contains(
                search,
                StringComparison.OrdinalIgnoreCase));
    }

    return Results.Ok(products);
});
```

Адаптировать фильтрацию к выбранной предметной области.

Уровень 1 — 1 балл

Реализовать endpoint:

```
GET /api/products
```

Вернуть список объектов в формате JSON.

Уровень 2 — 2 балла

Реализовать endpoint:

```
GET /api/products/{id}
```

Вернуть объект по идентификатору.

Вернуть `404 Not Found` при отсутствии объекта.

Уровень 3 — 3 балла

Добавить query-параметр:

```
search
```

Реализовать фильтрацию объектов.

Проверить работу запроса:

```
GET /api/products?search=...
```

---

7. Блок 4. Реализовать POST-запрос

Создать endpoint:

```
POST /api/products
```

Реализовать получение объекта из тела HTTP-запроса.

Пример:

```
app.MapPost("/api/products", (Product product) =>
{
    ProductStore.Products.Add(product);

    return Results.Ok(product);
});
```

7.1. Реализовать автоматическую генерацию идентификатора

Не передавать идентификатор из клиента.

Определить новый идентификатор:

```
var newId = ProductStore.Products.Count == 0
    ? 1
    : ProductStore.Products.Max(x => x.Id) + 1;
```

Присвоить идентификатор новому объекту.

Добавить объект в коллекцию.

7.2. Вернуть статус 201 Created

Использовать:

```
return Results.Created(
    $"/api/products/{product.Id}",
    product);
```

Проверить получение:

```
201 Created
```

Уровень 1 — 1 балл

Создать endpoint:

```
POST /api/products
```

Принять объект из тела запроса.

Добавить объект в коллекцию.

Уровень 2 — 2 балла

Реализовать автоматическую генерацию `Id`.

Добавить созданный объект в общую коллекцию.

Уровень 3 — 3 балла

Вернуть статус:

```
201 Created
```

Указать URL созданного ресурса.

---

8. Блок 5. Реализовать PUT и DELETE

8.1. Реализовать PUT

Создать endpoint:

```
PUT /api/products/{id}
```

Найти объект по идентификатору.

Изменить его свойства.

Вернуть измененный объект.

Пример:

```
app.MapPut("/api/products/{id}", (int id, Product updatedProduct) =>
{
    var product = ProductStore.Products
        .FirstOrDefault(x => x.Id == id);

    if (product == null)
        return Results.NotFound();

    product.Name = updatedProduct.Name;
    product.Category = updatedProduct.Category;
    product.Price = updatedProduct.Price;
    product.IsAvailable = updatedProduct.IsAvailable;

    return Results.Ok(product);
});
```

8.2. Реализовать DELETE

Создать endpoint:

```
DELETE /api/products/{id}
```

Найти объект по идентификатору.

Удалить объект из коллекции.

Вернуть:

```
204 No Content
```

Пример:

```
app.MapDelete("/api/products/{id}", (int id) =>
{
    var product = ProductStore.Products
        .FirstOrDefault(x => x.Id == id);

    if (product == null)
        return Results.NotFound();

    ProductStore.Products.Remove(product);

    return Results.NoContent();
});
```

8.3. Выполнить полный CRUD

Последовательно выполнить:

```
POST
  ↓
GET
  ↓
PUT
  ↓
GET
  ↓
DELETE
  ↓
GET
```

Проверить изменение состояния коллекции после каждой операции.

Уровень 1 — 1 балл

Реализовать:

```
PUT /api/products/{id}
```

Уровень 2 — 2 балла

Реализовать:

```
DELETE /api/products/{id}
```

Вернуть `204 No Content` после успешного удаления.

Уровень 3 — 3 балла

Выполнить полный цикл CRUD.

Проверить создание, получение, изменение и удаление объекта.

---

9. Блок 6. Настроить HTTP-статусы и OpenAPI

9.1. Использовать HTTP-статусы

Использовать следующие статусы:

Ситуация	Статус
Успешный GET	`200 OK`
Успешный POST	`201 Created`
Успешный DELETE	`204 No Content`
Некорректные данные	`400 Bad Request`
Объект не найден	`404 Not Found`

9.2. Создать описание API

Добавить пакет:

```
dotnet add package Microsoft.AspNetCore.OpenApi
```

Добавить регистрацию OpenAPI:

```
builder.Services.AddOpenApi();
```

Добавить endpoint OpenAPI:

```
if (app.Environment.IsDevelopment())
{
```

```
    app.MapOpenApi();
}
```

Запустить приложение.

Проверить формирование OpenAPI-документа.

При необходимости установить Swagger UI и подключить графический интерфейс для тестирования API.

9.3. Создать документацию API

Создать или дополнить файл:

```
README.md
```

Добавить таблицу:

Метод	URL	Назначение
GET	`/api/products`	получить список
GET	`/api/products/{id}`	получить объект
POST	`/api/products`	создать объект
PUT	`/api/products/{id}`	изменить объект
DELETE	`/api/products/{id}`	удалить объект

Добавить примеры JSON-запросов и ответов.

Уровень 1 — 1 балл

Создать таблицу API в `README.md`.

Уровень 2 — 2 балла

Использовать корректные HTTP-статусы:

```
200
201
204
400
404
```

Уровень 3 — 3 балла

Настроить OpenAPI.

Проверить описание созданных endpoints.

При наличии Swagger UI выполнить тестирование API через графический интерфейс.

---

10. Блок 7. Создать Docker-образ и опубликовать API на Render

10.1. Создать Dockerfile

Создать в корне проекта файл:

```
Dockerfile
```

Использовать многоэтапную сборку:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

WORKDIR /src

COPY . .

RUN dotnet restore

RUN dotnet publish -c Release
    -o /app/publish
    /p:UseAppHost=false


FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final

WORKDIR /app

COPY --from=build /app/publish .

EXPOSE 10000

ENTRYPOINT ["dotnet", "wt-lab6-6minimalapi-ivanov.dll"]
```

Заменить имя DLL на имя DLL собственного проекта.

10.2. Настроить порт приложения

Получить номер порта из переменной окружения `PORT`:

```
var port = Environment.GetEnvironmentVariable("PORT")
           ?? "10000";

app.Run($"http://0.0.0.0:{port}");
```

Использовать `0.0.0.0` в качестве адреса прослушивания приложения.

10.3. Проверить Docker-сборку

Создать Docker-образ:

```
docker build -t wt-lab6-6minimalapi .
```

Запустить контейнер:

```
docker run -p 10000:10000 wt-lab6-6minimalapi
```

Проверить:

```
http://localhost:10000/
```

Проверить:

```
http://localhost:10000/api/products
```

10.4. Создать Git-репозиторий

Инициализировать Git:

```
git init
```

Добавить файлы:

```
git add .
```

Создать commit:

```
git commit -m "Create Minimal API"
```

Создать удаленный репозиторий с именем:

```
wt-lab6-6minimalapi-ivanov
```

Подключить удаленный репозиторий:

```
git remote add origin <URL-репозитория>
```

Создать ветку `main`:

```
git branch -M main
```

Отправить проект:

```
git push -u origin main
```

Создать не менее трех осмысленных commit.

10.5. Создать Web Service на Render

Открыть Render Dashboard.

Выбрать:

```
New → Web Service
```

Подключить GitHub.

Выбрать репозиторий:

```
wt-lab6-6minimalapi-ivanov
```

Выбрать ветку:

```
main
```

Выбрать среду:

```
Docker
```

Запустить deployment.

Проверить:

- Build Logs;
- Deploy Logs;
- статус сервиса;
- запуск Docker-контейнера;
- доступность приложения.

Render предоставляет опубликованному Web Service публичный адрес вида:

```
https://<имя-сервиса>.onrender.com
```

10.6. Проверить публичный API

Проверить endpoint:

```
GET https://<имя-сервиса>.onrender.com/
```

Проверить:

```
GET https://<имя-сервиса>.onrender.com/api/info
```

Проверить:

```
GET https://<имя-сервиса>.onrender.com/api/products
```

Проверить:

```
GET https://<имя-сервиса>.onrender.com/api/products/1
```

Выполнить `POST`, `PUT` и `DELETE` через Postman, Swagger UI, VS Code REST Client или `curl`.

10.7. Выполнить повторный deployment

Изменить код приложения.

Создать новый commit:

```
git add .
git commit -m "Update API"
```

Отправить изменения:

```
git push
```

Дождаться завершения нового deployment на Render.

Проверить работу измененного API по публичному URL.

Добавить в `README.md`:

```
GitHub:
https://github.com/...

Render:
https://....onrender.com
```

Уровень 1 — 1 балл

Выполнить:

- инициализацию Git;
- создание не менее трех commit;
- создание Dockerfile;
- сборку Docker-образа.

Уровень 2 — 2 балла

Опубликовать Minimal API на Render.

Получить публичный HTTPS-адрес.

Проверить endpoint:

```
GET /api/products
```

по публичному адресу.

Уровень 3 — 3 балла

Изменить код опубликованного приложения.

Создать новый commit.

Выполнить `git push`.

Дождаться повторного deployment на Render.

Проверить обновленное приложение по публичному URL.

Проверить основные REST endpoints после повторного deployment.

---

11. Итоговая структура проекта

Получить следующую структуру:

```
wt-lab6-6minimalapi-ivanov/
│
├── Models/
│   └── Product.cs
│
├── Data/
│   └── ProductStore.cs
│
├── Program.cs
│
├── Dockerfile
├── README.md
├── .gitignore
│
└── wt-lab6-6minimalapi-ivanov.csproj
```

---

12. Итоговый набор API

Реализовать следующие endpoints:

Метод	Endpoint	Назначение
GET	`/`	проверить работу приложения
GET	`/api/info`	получить информацию о приложении
GET	`/api/products`	получить список объектов
GET	`/api/products/{id}`	получить объект
GET	`/api/products?search=...`	выполнить поиск
POST	`/api/products`	создать объект
PUT	`/api/products/{id}`	изменить объект
DELETE	`/api/products/{id}`	удалить объект

---

13. Тестирование API

Проверить API с использованием одного или нескольких инструментов:

- браузер;
- Postman;
- Swagger UI;
- VS Code REST Client;
- `curl`.

Проверить следующие операции:

```
GET /api/products
GET /api/products/1
GET /api/products/999
POST /api/products
PUT /api/products/1
DELETE /api/products/1
```

Проверить получение соответствующих HTTP-статусов.

Проверить работу API:

1. локально;
2. в Docker-контейнере;
3. после публикации на Render.

---

14. Отчет

Подготовить отчет в формате PDF.

Использовать имя:

```
wt-lab6-ivanov.pdf
```

Включить в отчет:

1. титульный лист;
2. цель работы;
3. выбранную предметную область;
4. название проекта;
5. структуру проекта;
6. код модели;
7. код хранилища данных;
8. код GET endpoints;
9. код POST endpoint;
10. код PUT endpoint;
11. код DELETE endpoint;
12. таблицу API;
13. примеры JSON-запросов;
14. примеры JSON-ответов;
15. HTTP-статусы;
16. OpenAPI;
17. Dockerfile;
18. результат Docker-сборки;
19. GitHub-репозиторий;
20. Render Dashboard;
21. результат deployment;
22. скриншот работающего API;
23. публичный Render URL;
24. результат повторного deployment;
25. результаты тестирования;
26. вывод.

---

15. Контрольные вопросы

1. Что такое Minimal API?
2. Чем Minimal API отличается от MVC?
3. Чем Minimal API отличается от Razor Pages?
4. Что такое endpoint?
5. Что такое REST API?
6. Что такое HTTP API?
7. Какие HTTP-методы использовать для CRUD?
8. Для чего использовать `GET`?
9. Для чего использовать `POST`?
10. Для чего использовать `PUT`?
11. Для чего использовать `DELETE`?
12. Что такое параметр маршрута?
13. Что такое query-параметр?

14. Что такое тело HTTP-запроса?
15. Что такое JSON?
16. Что означает `200 OK`?
17. Что означает `201 Created`?
18. Что означает `204 No Content`?
19. Что означает `400 Bad Request`?
20. Что означает `404 Not Found`?
21. Для чего использовать `Results.Ok()`?
22. Для чего использовать `Results.Created()`?
23. Для чего использовать `Results.NotFound()`?
24. Для чего использовать `Results.NoContent()`?
25. Для чего использовать `ProductStore`?
26. Почему использовать одну коллекцию для всех endpoints?
27. Что такое OpenAPI?
28. Для чего использовать Swagger UI?
29. Что такое Docker?
30. Для чего использовать Dockerfile?
31. Для чего использовать многоэтапную сборку Docker-образа?
32. Почему настроить приложение на прослушивание `0.0.0.0`?
33. Для чего использовать переменную окружения `PORT`?
34. Что такое Render Web Service?
35. Как связать Git-репозиторий с Render?
36. Что происходит после выполнения `git push`?
37. Как проверить работу API после deployment?
38. Где хранить данные текущего приложения?
39. Почему данные `ProductStore` не являются постоянным хранилищем?
40. Какие изменения потребуются для перехода от `ProductStore` к PostgreSQL?

---

16. Оценивание

Максимальное количество — **21 балл**.

Блок	1 балл	2 балла	3 балла
1. Создать Minimal API	создать проект	создать `/`	создать `/api/info`
2. Создать модель и данные	создать модель	расширить модель и данные	создать общий `ProductStore`
3. Реализовать GET	реализовать список	реализовать получение по ID	реализовать фильтрацию
4. Реализовать POST	создать объект	создать автоматический ID	вернуть `201 Created`
5. Реализовать PUT/DELETE	реализовать PUT	реализовать DELETE	выполнить полный CRUD

Блок	1 балл	2 балла	3 балла
6. Настроить HTTP/ OpenAPI	создать документацию	использовать статусы	настроить OpenAPI
7. Docker + Git + Render	создать Dockerfile и Git	опубликовать на Render	выполнить повторный deployment

Максимум: 21 балл.

---

17. Результат работы

Получить работающий REST API на базе ASP.NET Core Minimal API.

Обеспечить:

- выполнение CRUD-операций;
- передачу данных в JSON;
- использование HTTP-статусов;
- описание API через OpenAPI;
- сборку приложения в Docker;
- хранение проекта в Git-репозитории;
- публикацию приложения на Render;
- доступ к API по публичному HTTPS-адресу.